

画 (painting) 题解

首先把整个字符矩阵按题目要求分块，问题相当于给一个元素是大小为 3×3 、字符集为 R, G, B 的字符矩阵的长度为 $\frac{nm}{9}$ 的数组去重。

做法有很多。可以把小字符矩阵看做一个三进制数，也可以直接用 string 存储然后排序+去重（C++ 里 string 有自带的比较关系）。复杂度视实现而定。

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7      int n, m;
8      if (!(cin >> n >> m)) return 0;
9      vector<string> g(n);
10     for (int i = 0; i < n; ++i) cin >> g[i];
11
12     const int MAXCODE = 19683; // 3^9
13     vector<char> seen(MAXCODE, 0);
14
15     auto mapColor = [](char c) -> int {
16         if (c == 'R') return 0;
17         if (c == 'G') return 1;
18         return 2; // 'B'
19     };
20
21     int H = n / 3, W = m / 3;
22     int distinct = 0;
23     for (int bi = 0; bi < H; ++bi) {
24         for (int bj = 0; bj < W; ++bj) {
25             int code = 0;
26             int baseRow = bi * 3;
27             int baseCol = bj * 3;
28             for (int di = 0; di < 3; ++di) {
29                 for (int dj = 0; dj < 3; ++dj) {
30                     code = code * 3 + mapColor(g[baseRow + di][baseCol +
31                     dj]);
32                 }
33             }
34             if (!seen[code]) {
35                 seen[code] = 1;
36                 ++distinct;
37             }
38         }
39     }
40     cout << distinct << '\n';
41     return 0;
42 }
```

书 (book) 题解

每本书的两种选择其实是唬人的，我们可以默认每本书都丢弃，然后回收第 i 本书的收益是 $a_i - b_i$ ，这样问题就变成了在 n 个数中选最多 m 个数使得和最大，排序后贪心即可。

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      ios::sync_with_stdio(false);
5      cin.tie(nullptr);
6      int n, m;
7      if (!(cin >> n >> m)) return 0;
8      vector<long long> a(n), b(n);
9      for (int i = 0; i < n; ++i) cin >> a[i];
10     for (int i = 0; i < n; ++i) cin >> b[i];
11
12     long long base = 0;
13     vector<long long> diff; diff.reserve(n);
14     for (int i = 0; i < n; ++i) {
15         base += b[i];
16         long long d = a[i] - b[i];
17         if (d > 0) diff.push_back(d);
18     }
19     //一开始默认所有书都是不带走的
20     if ((int)diff.size() > m) {
21         nth_element(diff.begin(), diff.begin() + m, diff.end(), greater<long
22         long>());
23         //快速排序
24         //只需要知道前m大
25
26         diff.resize(m);
27     }
28     long long extra = 0;
29     for (auto v : diff) extra += v;
30
31     cout << (base + extra) << '\n';
32     return 0;
33 }
```

琴 (instrument) 题解

a_i 最大只有 50，容易想到对每次询问直接枚举值域。问题变成快速求出区间 $[l, r]$ 中 x 的出现次数，也就是 $\sum_{i=l}^r [a_i == x]$ ，对每个 x 记 $S(x, i) = \sum_{j=1}^i [a_j == x]$ ，也就是每个值开一个前缀和数组，则出现次数为 $S(x, r) - S(x, l - 1)$ 。

令 V 为值域范围，我们先以 $O(nV)$ 的复杂度预处理出 S 数组，对于每个询问从 1 枚举到 V ，先找到最小的出现在 $[l, r]$ 里的数，再计算答案。复杂度 $O((n + m)V)$ 。

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
```

```

6      cin.tie(nullptr);
7
8      int n;
9      if (!(cin >> n)) return 0;
10     vector<int> a(n + 1);
11     for (int i = 1; i <= n; ++i) cin >> a[i];
12     const int MAX_V = 50;
13     vector<vector<int>> cnt(MAX_V + 1, vector<int>(n + 1, 0));
14     for (int v = 1; v <= MAX_V; ++v) {
15         for (int i = 1; i <= n; ++i) {
16             cnt[v][i] = cnt[v][i - 1] + (a[i] == v);
17         }
18     }
19
20     int m;
21     cin >> m;
22     while (m--) {
23         int l, r;
24         cin >> l >> r;
25
26         int total = 0, mn = 0;
27         for (int v = 1; v <= MAX_V; ++v) if (cnt[v][r] > cnt[v][l - 1]) {
28             mn = v;
29             break;
30         }
31         for (int v = mn; v <= MAX_V; ++v) {
32             int cur = cnt[v][r] - cnt[v][l - 1];
33             if (cur == 0) break;
34             total += cur;
35         }
36         cout << total << '\n';
37     }
38     return 0;
39 }

```

棋 (chess) 题解

这类问题有一个通用的算法是状压 dp，把计算贡献要用到的格子的状态都记下来。在这里就是 $f(i, s_1, s_2)$ 表示前 i 列均已确定颜色，同时第 $i, i - 1$ 列的染色状态分别为 s_1', s_2' 所能获得的最大答案。

但是你会发现这样子一个格子可能被计算多次贡献，为了防止这一点，有两种方法。第一种是本着“哪些信息影响后续决策就记录哪些信息”的思想，多记二维表示第 $i, i - 1$ 列的每个格子是否已经算在了答案里，这样枚举下一列的染色状态就能转移了。第二种是一个很巧妙的手法，那就是在状态里记录前 4 列的染色状态，转移时枚举下一列，这样得到一个 3×5 的矩阵，我们只计算中间那一列格子的贡献，这样就不会算重了。这两种算法的状态都是 $2^{12} \times n$ 级别。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4  const ll NEG_INF = -(1LL << 60);
5
6  inline int bit(int x, int k) { return (x >> k) & 1; }
7

```

```

8  /* 鐘舵€佺紕鐮佺粗
9      | colB_colored(3bit) | colB_pattern(3bit) | colA_colored(3bit) |
      colA_pattern(3bit) |
10     鏈€鑾€寔寔鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€ */
11     inline int encode(int pa, int ca, int pb, int cb) {
12         return pa | (ca << 3) | (pb << 6) | (cb << 9);
13     }
14     inline void decode(int s, int &pa, int &ca, int &pb, int &cb) {
15         pa = s & 7;
16         ca = (s >> 3) & 7;
17         pb = (s >> 6) & 7;
18         cb = (s >> 9) & 7;
19     }
20
21     int main() {
22         // freopen(".out", "w", stdout);
23         int n;
24         if (!(cin >> n)) return 0;
25         vector<vector<int>> w(3, vector<int>(n)); // w[row][col]
26         for (int r = 0; r < 3; ++r)
27             for (int c = 0; c < n; ++c)
28                 cin >> w[r][c];
29
30         /* n = 1 鎯€2鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€ */
31         if (n <= 2) {
32             ll best = NEG_INF;
33             for (int p0 = 0; p0 < 8; ++p0) {
34                 for (int p1 = 0; p1 < (n == 2 ? 8 : 1); ++p1) {
35                     ll cur = 0;
36                     /* 鑼€?0 */
37                     if (p0 == 0 || p0 == 7) {
38                         int sgn = (p0 & 1) ? 1 : -1;
39                         for (int r = 0; r < 3; ++r) cur += 1LL * w[r][0] * sgn;
40                     }
41                     if (n == 2 && (p1 == 0 || p1 == 7)) {
42                         int sgn = (p1 & 1) ? 1 : -1;
43                         for (int r = 0; r < 3; ++r) cur += 1LL * w[r][1] * sgn;
44                     }
45                     best = max(best, cur);
46                 }
47             }
48             cout << best << '\n';
49             return 0;
50         }
51
52         /* n >= 3鑼€鑼€P */
53         const int SZ = 4096;
54         vector<ll> dp(SZ, NEG_INF), ndp(SZ, NEG_INF);
55
56         /* 鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€鑼€ */
57         for (int p0 = 0; p0 < 8; ++p0)
58             for (int p1 = 0; p1 < 8; ++p1) {
59                 int s = encode(p0, 0, p1, 0);
60                 dp[s] = 0;
61             }
62

```

```

63  /* 涓诲惊整細澶動慙錄?idx = j (0-based), j 湑?2 錄?n-1 */
64  for (int j = 2; j < n; ++j) {
65      int idxFinal = j - 2; // 鏈疆瑕估晶鏢嬌殞錄?
66      fill(ndp.begin(), ndp.end(), NEG_INF);
67
68      for (int st = 0; st < SZ; ++st) if (dp[st] > NEG_INF) {
69          int pa, ca, pb, cb;
70          decode(st, pa, ca, pb, cb);
71
72          // printf("%d:%d,%d,%d,%d\n", j, pa, ca, pb, cb);
73
74          for (int pc = 0; pc < 8; ++pc) { // 鑑板坵鐸嚟戮
75              int ca2 = ca, cb2 = cb, cc = 0;
76              ll gain = 0;
77
78              auto add_cell = [&](int row, int sym, int &cflag, const int
79  *wcol, bool score_now) {
80                  if (!(cflag & (1 << row))) {
81                      if (score_now)
82                          gain += 1LL * wcol[row] * (sym ? +1 : -1);
83                      cflag |= (1 << row);
84                  }
85              };
86
87              const int wA[] = {w[0][idxFinal], w[1][idxFinal], w[2]
88  [idxFinal]}; // 鍑富€煎坵鐸固拏
89
90              for (int r = 0; r < 3; ++r) if (bit(ca, r)) gain += wA[r] *
91  (bit(pa, r) ? +1 : -1);
92
93              /* 1. 縮靛惺涓�綰綰匡紙鏻綰坵坵鐸稿惺綰? */
94              if (pa == 0 || pa == 7) {
95                  int sym = pa & 1;
96                  for (int r = 0; r < 3; ++r)
97                      add_cell(r, sym, ca2, wA, true);
98              }
99
100              /* 2. 妯惺涓�綰綰匡紙涓�錫勳嚟錄ゆ杵綰? */
101              for (int r = 0; r < 3; ++r) {
102                  int s1 = bit(pa, r), s2 = bit(pb, r), s3 = bit(pc, r);
103                  if (s1 == s2 && s2 == s3) {
104                      add_cell(r, s1, ca2, wA, true); // (r, colA)
105                      add_cell(r, s1, cb2, wA, false); // (r, colB) (閱
106  確嘶湑ユ悝緇擇睦)
107                      cc |= (1 << r); // (r, colC)
108                  }
109              }
110
111              /* 3. 漢硅綰綰?欵?(0,1,2) */
112              if (bit(pa, 0) == bit(pb, 1) && bit(pb, 1) == bit(pc, 2)) {
113                  int sym = bit(pa, 0);
114                  add_cell(0, sym, ca2, wA, true); // (0, colA)
115                  add_cell(1, sym, cb2, wA, false); // (1, colB)
116                  cc |= (1 << 2); // (2, colC)
117              }

```

```

115         /* 4. 漢語? (2,1,0) */
116         if (bit(pa, 2) == bit(pb, 1) && bit(pb, 1) == bit(pc, 0)) {
117             int sym = bit(pa, 2);
118             add_cell(2, sym, ca2, wA, true); // (2, colA)
119             add_cell(1, sym, cb2, wA, false); // (1, colB)
120             cc |= (1 << 0); // (0, colC)
121         }
122
123         int st2 = encode(pb, cb2, pc, cc);
124         ndp[st2] = max(ndp[st2], dp[st] + gain);
125         // printf("(%d,%d,%d,%d):%lld\n",pb,cb2,pc,cc,ndp[st2]);
126     }
127 }
128 dp.swap(ndp);
129 }
130
131 /* 鎢蹠熬鑄氫鎢鎢+堦 n-2 涓?n-1鑄岯或鎢?迨鎢虹幫欽滅旱錫尧€漚竺杓疼壕 */
132 ll ans = NEG_INF;
133 for (int st = 0; st < SZ; ++st) if (dp[st] > NEG_INF) {
134     int pa, ca, pb, cb;
135     decode(st, pa, ca, pb, cb);
136     ll val = dp[st];
137
138     /* 錄?n-2 */
139     for (int r = 0; r < 3; ++r) if (bit(ca, r)) val += 1LL * w[r][n - 2]
140 * (bit(pa, r) ? +1 : -1);
141     if (pa == 0 || pa == 7) {
142         int sym = pa & 1;
143         for (int r = 0; r < 3; ++r)
144             if (!(ca & (1 << r)))
145                 val += 1LL * w[r][n - 2] * (sym ? +1 : -1);
146     }
147     /* 錄?n-1 */
148     for (int r = 0; r < 3; ++r) if (bit(cb, r)) val += 1LL * w[r][n - 1]
149 * (bit(pb, r) ? +1 : -1);
150     if (pb == 0 || pb == 7) {
151         int sym = pb & 1;
152         for (int r = 0; r < 3; ++r)
153             if (!(cb & (1 << r)))
154                 val += 1LL * w[r][n - 1] * (sym ? +1 : -1);
155     }
156     // if(val > 100) printf("(%d,%d,%d,%d) %lld
157 %lld\n",pa,ca,pb,cb,dp[st],val);
158     ans = max(ans, val);
159 }
160
161 cout << ans << '\n';
162 return 0;
163 }
164
165 /*
166 4
167 1 2 3 4
168 4 -100 2 1
169 1 3 1 3
170 -----
171 119

```

20pts

$f[i][j]$ 表示 $b_i = j$ 的情况下，填入 b 序列的前 i 项的方案数。

那么 $f[i][j]$ 可以转移到 $f[i+1][k]$ 当且仅当 $j \leq b_i, k \leq b_i, j \neq k$ ，假设值域为 V ，时间复杂度为 $O(nV)$ 。

40pts

首先直觉告诉我们合法方案数不好求，所以正难则反考虑求不合法的情况，不合法的情况也不好求，但是我们可以比较自然地想到容斥。

也就是求 $F(i)$ 表示至少有 i 个坏点的方案数。

然后考虑动态规划，设 $f_{i,j}$ 表示前 i 项划分成 j 部分，部分内部是相同的数字，但是相邻的部分不要求不同（因为你是容斥，保证至少多少个不同即可）。

显然 $F(i) = f_{n,n-i}$ 。

我们发现这个东西和最小值有关系，一个部分的取值方案就是最小值，这点比较显然。

转移为

$$f_{i,j} = \sum_{k=1}^{i-1} f_{k-1,j-1} \times \min_{x=k}^j a_x$$

时间复杂度为 $O(n^3)$ 。

60pts

我们并不关心 j 的具体大小，只关心 j 的奇偶性，因为容斥系数为 $(-1)^{n-j}$ ，所以每取一段，就让方案数乘上 -1 ，我们可以忽略掉 j 这一维

$$f_i = \sum_{k=1}^i -f_{k-1} \times \min_{x=k}^j a_x$$

时间复杂度为 $O(n^2)$ 。

100pts

考虑怎么维护 $\min_{x=k}^j a_x$ ，可以维护单调栈的具体形态，利用前缀和在单调栈变化时进行 DP 转移，时间复杂度为 $O(n)$ 。

```

1  #include <bits/stdc++.h>
2  #define int long long
3  #pragma GCC optimize("Ofast", "inline", "-ffast-math")
4  #pragma GCC target("avx,sse2,sse3,sse4,mmx")
5  using namespace std;
6
7  const int mod = 998244353;
8  const int N = 1e6+5;
9  int f[N][2], g[N][2];
10 int n, a[N], stk[N], top;
11
```

```

12 int M(int x){
13     return (x%mod+mod)%mod;
14 }
15
16 signed main(){
17     ios::sync_with_stdio(false);
18     cin.tie(0);cout.tie(0);
19     cin>>n;
20     for(int i=1;i<=n;i++){
21         cin>>a[i];
22     }
23     f[0][0]=1;f[0][1]=0;
24     g[0][0]=1;g[0][1]=0;
25     for(int i=1;i<=n;i++){
26         while(top && a[stk[top]] >= a[i]) top--;
27         int cur = top?stk[top]:0;
28         stk[++top]=i;
29         if(!cur){
30             f[i][0]=M(g[i-1][1]*a[i]);
31             f[i][1]=M(g[i-1][0]*a[i]);
32         }
33         else{
34             f[i][0]=M(f[cur][0]+M(M(g[i-1][1]-g[cur-1][1]))*a[i]));
35             f[i][1]=M(f[cur][1]+M(M(g[i-1][0]-g[cur-1][0]))*a[i]));
36         }
37         g[i][0] = M(g[i-1][0] + f[i][0]);
38         g[i][1] = M(g[i-1][1] + f[i][1]);
39     }
40     if(n&1) cout<<(M(f[n][1]-f[n][0]))<<'\\n';
41     else cout<<(M(f[n][0]-f[n][1]))<<'\\n';
42     return 0;
43 }

```